Jupyter Notebook

External Oral Examination – Code Explanation & Viva Notes

Five practicals covered
1. Uber fare prediction and data analysis
2. Email spam classification using KNN and SVM
3. Customer churn prediction using ANN (Keras)
4. Diabetes prediction using KNN
5. Sales data clustering using K-Means

Prepared as concise, oral-ready explanations. Code notes follow the uploaded practical notebook and focus on what each important statement does.

How to use these notes

For each practical, first learn the **aim**, then revise the code blocks in order. Be ready to explain the purpose of the libraries, preprocessing steps, model training, prediction and evaluation. The short viva answers at the end are designed for quick revision.

Note: These notes explain the notebook code and its reported results. Where the notebook contains a particular setting or result, it is described as shown in the notebook.

1. Uber fare prediction and data analysis

PDF pages 13–26 | Aim: Analyze Uber trip data, calculate trip distance and predict fare using Linear Regression.

A. Importing libraries

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

Code / library	Short explanation
matplotlib.pyplot	Used to create graphs and charts.
numpy	Used for mathematical and numerical operations.
pandas	Used to read and manipulate datasets.
as plt / np / pd	Creates short aliases for the libraries.

B. Loading and inspecting the dataset

```
uber = pd.read_csv('uber.csv')
uber.head()
uber.info()
uber.isnull().sum()
```

Statement	Short explanation
read_csv()	Reads the CSV file into a DataFrame.
uber	Variable storing the dataset.
head()	Displays the first five rows.
info()	Shows columns, data types, non-null counts and memory use.
isnull().sum()	Counts missing values in each column.

C. Removing unnecessary columns and missing values

```
uber_2 = uber.drop(['Unnamed: 0', 'key'], axis=1)
uber_2.dropna(axis=0, inplace=True)
uber_2.isnull().sum()
```

drop() removes unwanted columns. **axis=1** means columns; **axis=0** means rows. **dropna()** removes rows with missing values. **inplace=True** updates the DataFrame directly. The final statement checks whether missing values remain.

D. Haversine function

```
def haversine(lon_1, lon_2, lat_1, lat_2):
lon_1, lon_2, lat_1, lat_2 = map(np.radians,
[lon_1, lon_2, lat_1, lat_2])
diff_lon = lon_2 - lon_1
diff_lat = lat_2 - lat_1
# Haversine calculation uses Earth's radius: 6371 km
km = 2 * 6371 * np.arcsin(...)
return km
```

Defines a function to calculate geographical distance between pickup and drop-off coordinates. **np.radians()** converts degrees to radians; the longitude and latitude differences are calculated; the Haversine formula uses Earth's radius (6371 km); **return** sends the distance back. The ellipsis represents the remainder of the formula in the notebook.

E. Calculating distance

```
uber_2['Distance'] = haversine(
uber_2['pickup_longitude'], uber_2['dropoff_longitude'],
uber_2['pickup_latitude'], uber_2['dropoff_latitude'])
uber_2['Distance'] = uber_2['Distance'].astype(float).round(2)
```

Creates a new Distance column from pickup/drop-off coordinates. **astype(float)** converts values to decimal numbers; **round(2)** rounds distance to two decimal places.

F. Scatter plot and outlier removal

```
plt.scatter(uber_2['Distance'], uber_2['fare_amount'])
plt.xlabel('Distance')
plt.ylabel('fare_amount')
```

scatter() creates a scatter plot. X-axis is trip distance and Y-axis is fare amount. It helps inspect the relationship and spot unusual values. The notebook removes outlier records using conditions including distance above 60 km, zero distance, negative fare and fare above 100.

G. Date and time preprocessing

```
uber_2['pickup_datetime'] = pd.to_datetime(uber_2['pickup_datetime'])
uber_2['Year'] = uber_2['pickup_datetime'].apply(lambda time: time.year)
```

Expression	Purpose
pd.to_datetime()	Converts a column into datetime format.
.year	Extracts year.
.month	Extracts month.
.day	Extracts day.
.dayofweek	Extracts weekday number.
.hour	Extracts hour.

The notebook also creates weekday-related and pickup/drop-off coordinate combination columns using string conversion and concatenation.

H. Data visualization and grouping

Code / function	Short explanation
plt.bar()	Creates bar charts.
groupby()	Groups rows by one or more columns.
sum()	Adds grouped values or counts records, depending on the data.
groupby(['Year','Month'])	Groups trips by year and month.
plot(kind='line')	Creates a line graph to show monthly trends.
sns.heatmap()	Shows values with colours.
cmap	Sets the heatmap colour scheme.
linewidth	Sets cell border width.

```
import seaborn as sns
sns.heatmap(df_h, cmap='Reds', linewidth=0.3)
```

The heatmap is used to visualize average rides by hour and weekday.

I. Statistical analysis

```
import statistics as st
st.mean(...)
st.median(...)
st.stdev(...)
uber_2.corr(numeric_only=True)
```

Function	Meaning
st.mean()	Calculates average.
st.median()	Finds middle value.
st.stdev()	Calculates standard deviation.
corr(numeric_only=True)	Calculates correlations between numerical columns.

J. Linear Regression model

```
X = uber_2['Distance'].values.reshape(-1, 1)
y = uber_2['fare_amount'].values.reshape(-1, 1)
```

X is the independent input (distance); **y** is the dependent output (fare). **values** converts a column to an array. **reshape(-1,1)** makes it a two-dimensional column array.

```
from sklearn.preprocessing import StandardScaler
std = StandardScaler()
y_std = std.fit_transform(y)
X_std = std.fit_transform(X)
```

StandardScaler standardizes values using mean and standard deviation. **fit_transform()** learns scaling parameters and applies the transformation.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X_std, y_std, test_size=0.2, random_state=...)
from sklearn.linear_model import LinearRegression
l_reg = LinearRegression()
l_reg.fit(X_train, y_train)
l_reg.score(X_train, y_train)
l_reg.score(X_test, y_test)
y_pred = l_reg.predict(X_test)
```

train_test_split() divides data into training and testing sets; **test_size=0.2** means 20% testing data. **random_state** makes the split reproducible. **LinearRegression()** creates the model; **fit()** trains it; **score()** reports R^2 score; **predict()** predicts fares for test distances.

Evaluation metrics: MAE = average absolute prediction error; MSE = average squared error; RMSE = square root of MSE.
Final purpose: predict Uber fare using trip distance.

2. Email spam classification using KNN and SVM

PDF pages 27–30 | Aim: Classify emails as spam or not spam using machine learning.

A. Importing libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn import metrics
```

Library / function	Short explanation
pandas	Dataset handling.
numpy	Numerical operations.
seaborn / matplotlib	Data visualization and plotting.
warnings.filterwarnings('ignore')	Hides warning messages.
train_test_split	Divides data into training and testing sets.
SVC	Support Vector Classifier.
metrics	Model evaluation functions.

B. Loading and inspecting dataset

```
df = pd.read_csv('emails.csv')
df.head()
df.columns
df.isnull().sum()
```

Loads the email dataset; displays the first five records; lists column names; counts missing values.

C. Data cleaning and X/y separation

```
df.dropna(inplace=True)
df.drop(['Email No.'], axis=1, inplace=True)
X = df.drop(['Prediction'], axis=1)
y = df['Prediction']
```

dropna() removes rows with missing values. **drop()** removes the email ID column. **X** stores input features; **y** stores the target label Prediction.

D. Feature scaling

```
from sklearn.preprocessing import scale
X = scale(X)
```

scale() standardizes numerical features so they are on comparable scales.

E. Splitting the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.3, random_state=...)
```

Uses 70% for training and 30% for testing. random_state makes the split reproducible.

F. KNN classifier

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
metrics.accuracy_score(y_test, y_pred)
metrics.confusion_matrix(y_test, y_pred)
```

KNeighborsClassifier imports KNN. **n_neighbors=7** uses seven nearest neighbours. **fit()** trains the classifier; **predict()** predicts spam/not-spam labels; **accuracy_score()** calculates accuracy; **confusion_matrix()** compares actual and predicted classes.

G. SVM classifier

```
model = SVC(C=1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
metrics.accuracy_score(y_test, y_pred)
```

SVC creates a Support Vector Classifier. **C=1** is the regularization parameter controlling the trade-off between margin size and classification errors. The model is trained, used for prediction and evaluated.

The PDF reports approximately 93.8% SVM accuracy. Final purpose: compare KNN and SVM for email spam detection.

3. Customer churn prediction using ANN (Keras)

PDF pages 31–42 | Aim: Predict whether a customer will leave a company using an Artificial Neural Network.

A. Loading and inspecting dataset

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('Churn_Modelling.csv')
df.head()
df.shape
df.describe()
df.isnull()
df.isnull().sum()
df.info()
df.dtypes
df.columns
```

Code	Short explanation
read_csv()	Loads customer churn dataset.
head()	Shows first five rows.
shape	Returns number of rows and columns; notebook output is (10000, 14).
describe()	Shows mean, standard deviation, minimum, maximum and quartiles.
isnull() / isnull().sum()	Checks and counts missing values.
info()	Shows data types and non-null counts.
dtypes	Shows each column's data type.
columns	Lists column names.

B. Removing unnecessary columns

```
df = df.drop(['RowNumber', 'Surname', 'CustomerId'], axis=1)
```

Removes row number, surname and customer ID because they are identification fields rather than useful customer behaviour predictors.

C. Data visualization

```
def visualization(x, y, xlabel):
plt.figure(figsize=(10,5))
plt.hist([x, y], color=['red', 'green'], label=['exit', 'not_exit'])
plt.xlabel(xlabel)
plt.ylabel('No. of customers')
plt.legend()
```

Defines a reusable graph function. **figure(figsize=)** sets graph size; **hist()** creates a histogram; xlabel/ylabel name the axes; legend displays category labels.

```
df_churn_exited = df[df['Exited']==1]['Tenure']
df_churn_not_exited = df[df['Exited']==0]['Tenure']
```

Filters tenure values for customers who exited (1) and customers who stayed (0). The notebook repeats this comparison for customer age.

D. Converting categorical variables

```
states = pd.get_dummies(df['Geography'], drop_first=True)
gender = pd.get_dummies(df['Gender'], drop_first=True)
df = pd.concat([df, gender, states], axis=1)
```

get_dummies() converts categorical values into numerical indicator columns. **drop_first=True** drops one category to avoid redundant dummy columns. **concat(axis=1)** joins columns side by side.

Why dummy variables? Most machine learning models require numerical input, so categorical data is encoded.

E. Define input and target

```
X = df.drop(['Exited'], axis=1)
y = df['Exited']
```

X contains input features; y contains the target. Exited=1 means customer left; Exited=0 means customer stayed.

F. Train/test split and standardization

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.30)
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Splits into 70% training and 30% testing. StandardScaler scales numerical features. **fit_transform()** learns parameters from training data and scales it. **transform()** applies those same parameters to test data to avoid data leakage.

G. Building ANN using Keras

```
import keras
from keras.models import Sequential
from keras.layers import Dense
classifier = Sequential()
classifier.add(Dense(activation='relu', input_dim=11, units=6,
kernel_initializer='uniform'))
classifier.add(Dense(activation='relu', units=6,
kernel_initializer='uniform'))
classifier.add(Dense(activation='sigmoid', units=1,
kernel_initializer='uniform'))
```

Code / parameter	Short explanation
Keras	High-level deep learning API.
Sequential()	Creates a model with layers in sequence.
Dense	Fully connected neural network layer.
input_dim=11	Specifies 11 input features.
units=6	Six neurons in the hidden layer.
activation='relu'	ReLU activation; outputs zero for negative inputs.
activation='sigmoid'	Produces output between 0 and 1 for binary classification.
kernel_initializer='uniform'	Initializes weights using a uniform distribution.

H. Compile, train and evaluate

```
classifier.compile(optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy'])
classifier.summary()
classifier.fit(X_train, y_train, batch_size=10, epochs=50)
y_pred = classifier.predict(X_test)
y_pred = (y_pred > 0.5)
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
sns.heatmap(cm, annot=True)
print(classification_report(y_test, y_pred))
```

Code / term	Short explanation
adam	Optimizer that updates model weights.
binary_crossentropy	Loss function for binary classification.
accuracy	Measures fraction of correct predictions.
summary()	Displays model layers, output shapes and parameters.
fit()	Trains the neural network.

Code / term	Short explanation
batch_size=10	Processes 10 samples per batch.
epochs=50	Makes 50 complete passes through training data.
predict()	Produces churn probabilities.
> 0.5	Converts probabilities into binary predictions.
confusion_matrix()	Compares actual and predicted classes.
classification_report()	Shows precision, recall, F1-score and support.

The PDF reports approximately 83.43% accuracy. Final purpose: predict customers who are likely to leave the company.

4. Diabetes prediction using KNN

PDF pages 43–70 | Aim: Predict diabetes using medical attributes and the K-Nearest Neighbours algorithm.

A. Importing libraries and loading data

```
from mlxtend.plotting import plot_decision_regions
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
diabetes_data = pd.read_csv('diabetes.csv')
diabetes_data.head()
diabetes_data.info()
diabetes_data.describe()
diabetes_data.describe().T
```

Code / library	Short explanation
mlxtend.plotting	Provides decision-region visualization.
pandas / numpy	Data handling and numerical operations.
matplotlib / seaborn	Graph plotting and statistical visualization.
warnings.filterwarnings('ignore')	Hides warning messages.
read_csv()	Loads diabetes dataset.
head()	Displays first five records.
info()	Shows data structure and types.
describe()	Shows statistical summary.
describe().T	Transposes the summary table.

B. Handling missing or zero values

```
diabetes_data_copy = diabetes_data.copy(deep=True)
diabetes_data_copy.isnull().sum()
# Notebook handles zero values in selected medical columns
# using mean or median, then fills missing values with fillna().
```

Creates an independent copy. The notebook treats zero values in Glucose, BloodPressure, SkinThickness, Insulin and BMI as missing measurements. It uses mean for Glucose and BloodPressure, and median for SkinThickness, Insulin and BMI. **fillna()** replaces missing values.

Mean is the average. **Median** is the middle value in sorted data and is less affected by extreme values.

C. Data visualization

```
diabetes_data.hist(figsize=(20,20))
import missingno as msno
msno.bar(diabetes_data)
diabetes_data.Outcome.value_counts()
diabetes_data.Outcome.value_counts().plot(kind='bar')
from pandas.plotting import scatter_matrix
scatter_matrix(diabetes_data, figsize=(25,25))
sns.pairplot(diabetes_data_copy, hue='Outcome')
```

Function	Short explanation
hist()	Creates histograms showing distributions.
missingno.bar()	Visualizes missing-value patterns.
value_counts()	Counts occurrences of each outcome.
plot(kind='bar')	Creates a bar graph.
scatter_matrix()	Displays pairwise relationships between numerical columns.
pairplot(hue='Outcome')	Creates pairwise plots with colours by outcome class.

D. Correlation heatmap

```
sns.heatmap(diabetes_data_copy.corr(), annot=True, cmap='RdYlGn')
```

corr() calculates correlations. **annot=True** displays values in the cells. **cmap** selects the colour scheme. Correlation measures the strength and direction of a relationship between numerical variables.

E. StandardScaler and X/y

```
from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
X = pd.DataFrame(sc_X.fit_transform(
diabetes_data_copy.drop(['Outcome'], axis=1)))
y = diabetes_data.Outcome.copy()
```

StandardScaler standardizes input features. The Outcome target column is excluded from X. **y** stores the diabetes outcome. X represents medical attributes; y represents the class label.

F. Train/test split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=1/3, random_state=...)
```

Splits the data into approximately 66.7% training and 33.3% testing. random_state makes the split reproducible.

G. Finding the best K value

```
from sklearn.neighbors import KNeighborsClassifier
train_scores = []
test_scores = []
for i in range(1,15):
knn = KNeighborsClassifier(i)
knn.fit(X_train, y_train)
train_scores.append(knn.score(X_train, y_train))
test_scores.append(knn.score(X_test, y_test))
max_train_score = max(train_scores)
max_test_score = max(test_scores)
```

Imports KNN, creates lists to store scores, and tests K values from 1 to 14. Each model is trained and scored on training and testing data. **max()** finds the highest score.

H. Final KNN model and evaluation

```
knn = KNeighborsClassifier(11)
knn.fit(X_train, y_train)
knn.score(X_test, y_test)
y_pred = knn.predict(X_test)
confusion_matrix(y_test, y_pred)
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True)
```

The notebook selects K=11 as the final model. **score()** calculates test accuracy; **predict()** predicts diabetes outcomes; **confusion_matrix()** compares actual and predicted results; heatmap displays the matrix.

The notebook reports a maximum test score of approximately 76.56%.

I. Decision region visualization

```
plot_decision_regions(...)
```

Visualizes how the classifier separates the classes. Final purpose: classify patients into diabetes and non-diabetes categories using KNN.

5. Sales data clustering using K-Means

PDF pages 71–80 | Aim: Group sales records into three clusters using K-Means.

A. Importing libraries and loading data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
df = pd.read_csv('sales_data_sample.csv', sep=',', encoding='latin-1')
```

Code / library	Short explanation
pandas / numpy	Data manipulation and numerical operations.
matplotlib / seaborn	Graphs and statistical visualization.
KMeans	Imports K-Means clustering algorithm.
PCA	Imports Principal Component Analysis for dimensionality reduction.
read_csv()	Loads sales dataset.
sep=','	Specifies comma separator.
encoding='latin-1'	Reads text using Latin-1 character encoding.

B. Inspecting and cleaning data

```
df.head()
df.shape
df.describe()
df.info()
df.isnull().sum()
df.dtypes
```

head() shows first five rows. **shape** returns dimensions; notebook output is 2823 rows and 25 columns. **describe()** gives statistics. **info()** shows data types and non-null counts. **isnull().sum()** counts missing values. **dtypes** lists column types.

```
df_drop = ['ADDRESSLINE1', 'ADDRESSLINE2', 'STATUS',
'POSTALCODE', 'CITY', 'TERRITORY']
df = df.drop(df_drop, axis=1)
```

Creates a list of unnecessary columns and removes them using drop().

C. Checking categorical values and encoding

```
df['COUNTRY'].unique()
df['PRODUCTLINE'].unique()
df['DEALSIZE'].unique()
productline = pd.get_dummies(df['PRODUCTLINE'])
DealSize = pd.get_dummies(df['DEALSIZE'])
df = pd.concat([df, productline, DealSize], axis=1)
```

unique() returns distinct values. **get_dummies()** converts product categories and deal sizes into numerical indicator columns. **concat(axis=1)** combines columns side by side.

```
df = df.drop(['COUNTRY', 'PRODUCTLINE'], axis=1)
df['PRODUCTCODE'] = pd.Categorical(df['PRODUCTCODE']).codes
df.drop('ORDERDATE', axis=1, inplace=True)
```

Removes original categorical columns, converts PRODUCTCODE categories to integer codes, and removes ORDERDATE.

D. Elbow method

```
distortions = []
K = range(1,10)
for k in K:
kmeanModel = KMeans(n_clusters=k)
kmeanModel.fit(df)
distortions.append(kmeanModel.inertia_)
plt.plot(K, distortions, 'bx-')
plt.xlabel('k')
```

```
plt.ylabel('Distortion')
plt.title('The Elbow Method')
plt.show()
```

Statement	Short explanation
distortions = []	Creates a list to store inertia values.
range(1,10)	Tests cluster counts from 1 to 9.
KMeans(n_clusters=k)	Creates a model with k clusters.
fit(df)	Fits the model to the data.
inertia_	Sum of squared distances from points to their assigned centres.
plt.plot()	Plots cluster count against distortion/inertia.
plt.show()	Displays the graph.

The notebook observes an elbow around K=3, so it selects three clusters. The elbow method looks for the point after which adding more clusters reduces inertia more gradually.

E. Building K-Means model

```
X_train = df.values
model = KMeans(n_clusters=3, random_state=2)
model.fit(X_train)
predictions = model.predict(X_train)
unique, counts = np.unique(predictions, return_counts=True)
counts_df = pd.DataFrame(counts, columns=['Cluster1','Cluster2','Cluster3'])
```

values converts the DataFrame into a NumPy array. **n_clusters=3** requests three clusters. **random_state=2** makes initialization reproducible. **fit()** learns cluster centres. **predict()** assigns each record to a cluster. **np.unique(return_counts=True)** returns unique labels and their counts.

Cluster	Records shown in notebook
Cluster 1	1344
Cluster 2	398
Cluster 3	1081

F. PCA for visualization

```
pca = PCA(n_components=2)
reduced_x = pd.DataFrame(
pca.fit_transform(X_train), columns=['PCA1','PCA2'])
plt.scatter(reduced_x['PCA1'], reduced_x['PCA2'])
```

PCA(n_components=2) reduces the data to two principal components. **fit_transform()** learns the components and transforms the data. PCA1 and PCA2 are used to visualize the high-dimensional data on a two-dimensional scatter plot.

G. Cluster centres and final plot

```
model.cluster_centers_
reduced_centers = pca.transform(model.cluster_centers_)
plt.scatter(reduced_centers[:,0], reduced_centers[:,1],
color='black', marker='x')
reduced_x['Clusters'] = predictions
```

cluster_centers_ returns the learned cluster centres. **pca.transform()** maps those centres into PCA coordinates. The scatter plot marks centres with black X symbols. The Clusters column stores each row's assigned cluster label. The final plots use different colours to show the three groups.

Final purpose: group similar sales records into three clusters using K-Means.

Important viva questions – all five practicals

Notebook 1 – Uber fare prediction

Question	Short oral answer
What is Pandas?	A Python library for data manipulation.
What is a DataFrame?	A two-dimensional tabular data structure.
What is Haversine?	A formula to calculate distance between geographical coordinates.
What is an outlier?	A value that differs significantly from other values.
What is correlation?	It measures the relationship between two variables.
What is Linear Regression?	An algorithm that predicts continuous values.
What is StandardScaler?	It standardizes features using mean and standard deviation.
What is MAE?	Mean Absolute Error.
What is RMSE?	Root Mean Squared Error.
What is R^2 score?	It measures how well a regression model explains variation in the target.

Notebook 2 – Email spam classification

Question	Short oral answer
What is classification?	Predicting a category or class.
What is KNN?	K-Nearest Neighbours classifies using nearby data points.
What is SVM?	Support Vector Machine separates classes using a decision boundary.
What is K?	The number of nearest neighbours.
What is SVC?	Support Vector Classifier.
What is accuracy?	The ratio of correct predictions to total predictions.
What is a confusion matrix?	A table comparing actual and predicted classes.
What is spam?	Unwanted or unsolicited email.
What is feature scaling?	Bringing numerical features to comparable scales.

Notebook 3 – Customer churn prediction

Question	Short oral answer
What is churn?	When a customer stops using a company's service.
What is ANN?	Artificial Neural Network inspired by the human brain.
What is Keras?	A high-level deep learning API.
What is Sequential?	A model where layers are arranged sequentially.
What is Dense?	A fully connected neural network layer.
What is ReLU?	An activation function that outputs zero for negative values.
What is Sigmoid?	An activation function that produces values between 0 and 1.
What is Adam?	An optimization algorithm.
What is an epoch?	One complete pass through the training dataset.
What is batch size?	Number of samples processed before a weight update.
What is binary crossentropy?	A loss function for binary classification.

Notebook 4 – Diabetes prediction

Question	Short oral answer
What is diabetes prediction?	Predicting diabetes using medical attributes.
What is KNN?	Classification based on nearest data points.
What is Outcome?	The target variable representing diabetes status.
What is mean?	The average of values.
What is median?	The middle value of sorted data.
What is a histogram?	A graph showing frequency distribution.
What is pairplot?	A graph showing relationships between pairs of variables.
What is a heatmap?	A graph representing values using colours.
What is PCA?	Principal Component Analysis; it reduces dimensions.
What is a decision boundary?	A boundary separating different predicted classes.
Why use K=11?	The notebook selects 11 as its final K value.

Notebook 5 – Sales clustering

Question	Short oral answer
What is clustering?	Grouping similar data points together.
What is K-Means?	An unsupervised machine learning algorithm.
What is unsupervised learning?	Learning patterns without labelled target outputs.
What is a centroid?	The centre point of a cluster.
What is inertia?	Sum of squared distances from points to their cluster centres.
What is the Elbow method?	A method to select the number of clusters.
Why K=3?	The notebook observes an elbow around three clusters.
What is PCA?	A technique used to reduce dimensions.
What is n_clusters=3?	It creates three clusters.
What is fit()?	It fits or trains the model on the data.
What is predict()?	It assigns data points to clusters.

Most important differences to remember

Concept	Difference
Supervised vs Unsupervised	Supervised learning uses labelled data; unsupervised learning finds patterns without labels.
Classification vs Regression	Classification predicts categories; regression predicts continuous values.
KNN vs K-Means	KNN is supervised; K-Means is unsupervised.
ANN vs Traditional ML	ANN uses interconnected artificial neurons; traditional ML uses other learning algorithms.
Training vs Testing	Training teaches the model; testing evaluates it on unseen data.
Mean vs Median	Mean is the average; median is the middle value.
MAE vs MSE	MAE uses absolute errors; MSE squares the errors.
PCA vs StandardScaler	PCA reduces dimensions; StandardScaler standardizes feature values.

Quick memory tip: Uber = Linear Regression; Email = KNN + SVM; Churn = ANN; Diabetes = KNN; Sales = K-Means.

End of study notes